

ESE 499 Undergraduate Research –Summer 2026
Real-Time FM Audio Wireless Transceiver System Using PlutoSDR

Name: Sohee Kim

Student ID: 117643907

Date: June11, 2026

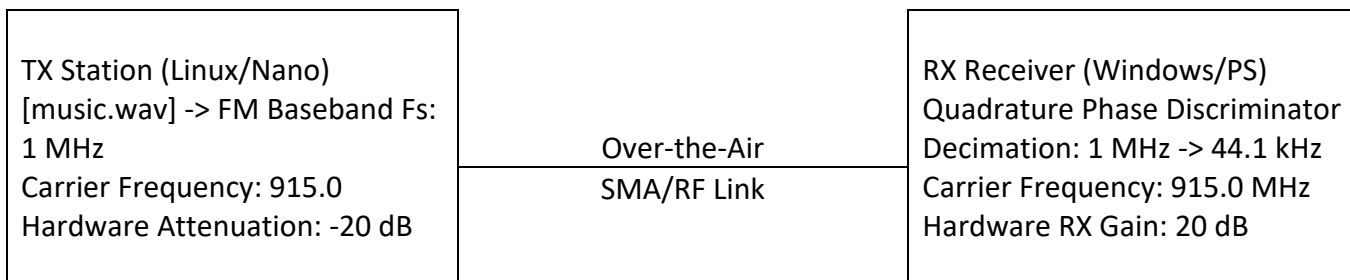
Email: sohee.kim.3@stonybrook.edu

Advisor: Prof. Harbans Dhadwal



1. System Architecture & Setup

This project implements a fully functional, over-the-air Frequency Modulation (FM) wireless audio broadcasting and reception loop using two Analog Devices ADALM-PLUTO (PlutoSDR) platforms. The transmitter runs on a Linux (Ubuntu) environment, while the receiver operates dynamically under Windows 11 via Python baseband signal processing pipelines.



Hardware Interconnection Specifications

- RF Link Type: Coaxial SMA Cable Direct Connection (Line-of-Sight Emulation)
- Port Mapping: Linux Transmitter TX1 Port $\rightarrow$ Windows Receiver RX1 Port
- Power & Gain Optimization: Adjusted to prevent analog-to-digital converter (ADC)

2. Transmitter (TX) Source Code

Filename: pluto_music_tx.py

Environment: Linux (Ubuntu) Terminal / nano editor

```
import numpy as np
import adi
import sys
from scipy.io import wavfile

# 1. Load the converted WAV file via SciPy
wav_filename = "music.wav"

try:
    print(f"[AUDIO] Reading '{wav_filename}' into DSP pipeline...")
    audio_fs, audio_data = wavfile.read(wav_filename)

    # Normalize integer PCM data to float ranges between -1.0 and 1.0
    if audio_data.dtype == np.int16:
        audio_data = audio_data / 32768.0
    elif audio_data.dtype == np.int32:
        audio_data = audio_data / 2147483648.0

    print(f"[AUDIO] Success! Loaded {len(audio_data)/audio_fs:.1f} seconds
of music.")
```



```

except FileNotFoundError:
    print(f"[ERROR] Could not find '{wav_filename}'.")
    print("[ERROR] Please run the ffmpeg command in your terminal first!")
    sys.exit(1)

# 2. Perform FM Modulation (Audio to Complex IQ Vector Mapping)
kf = 12000 # Frequency deviation constant matching the Windows RX specs
          exactly
phase = 2 * np.pi * kf * np.cumsum(audio_data) / audio_fs
tx_iq_signal = np.exp(1j * phase) * 0.5 # Safe hardware amplitude scaling
          to prevent distortion
tx_iq_signal = tx_iq_signal.astype(np.complex64)

# 3. Configure PlutoSDR Hardware Specifications for Continuous Broadcasting
try:
    sdr = adi.Pluto()
except Exception as e:
    print("[ERROR] Cannot find PlutoSDR hardware on Linux USB port.")
    sys.exit(1)

sdr.sample_rate = int(1e6) # TX Baseband Sampling Rate: 1 MHz
sdr.tx_lo = int(915e6) # Target RF Carrier Frequency: 915.0
MHz (ISM Band)
sdr.tx_hardwaregain_chan0 = -20 # Attenuated to -20 dB to prevent RX
saturation over direct cable link

print(f"\n[TX] 📡 Now broadcasting '{wav_filename}' over the air!")
print(f"[TX] 📡 Target Carrier Frequency: {sdr.tx_lo/1e6:.1f} MHz")
print("[TX] 📡 The music will loop continuously. Press Ctrl+C to shut down
your station.")

# 4. Push music IQ samples into PlutoSDR internal hardware buffer for
cyclic playback
try:
    sdr.tx_cyclic_buffer = True # Enable hardware memory buffering
loops for seamless streaming
    sdr.tx(tx_iq_signal) # Initiate broadcasting loop onto the
RF front-end

    while True:
        # Hold the execution thread open to sustain continuous cyclic
streaming
        pass

```

```
except KeyboardInterrupt:
    print("\n[TX] Shutting down the PlutoSDR radio station
    transmission...")
    sdr.tx_destroy_buffer()
```

3. Receiver (RX) Source Code

Filename: pluto_radio_rx.py

Environment: Windows 11 PowerShell

```
import os
import sys
import ctypes

# Force Windows system to extend its core DLL search path for libiio
binding
if sys.platform == "win32":
    try:
        # Bind standard Windows system DLL directory to prevent NoneType
        environment bugs
        ctypes.windll.kernel32.SetDllDirectoryW(None)
    except Exception:
        pass

import numpy as np
import adi
import sounddevice as sd

# 1. Initialize PlutoSDR Hardware Specifications for RX Receiver
try:
    sdr = adi.Pluto()
except Exception as e:
    print("[ERROR] Cannot find PlutoSDR hardware on Windows USB port.")
    print("[ERROR] Hardware binding failed. Please reconnect the USB
    cable.")
    sys.exit(1)

# Configure PlutoSDR RF parameters to match the transmitter (TX)
specifications
sdr.sample_rate = int(1e6)          # RX Baseband Sampling Rate: 1 MHz
sdr.rx_lo = int(915e6)              # Target RF Carrier Frequency: 915.0
MHz (ISM Band)
sdr.gain_control_mode_chan0 = 'manual'
```

```

sdr.rx_hardwaregain_chan0 = 20      # Attenuated to 20 dB to safely
digitize direct SMA input

# Define digital signal processing (DSP) and acoustic variables
audio_fs = 44100                    # Standard audio playback sampling rate
kf = 12000                          # Frequency deviation constant matching
TX specs
decimation_rate = int(sdr.sample_rate / audio_fs)

print(f"\n[RX] 📻 Radio Receiver Initialized at {sdr.rx_lo/1e6:.1f} MHz")
print("[RX] 📻 Real-time audio stream binding... Press Ctrl+C to mute.")

# 2. Open Continuous Audio Stream Buffer via Sounddevice (Non-blocking
Callback approach)
stream = sd.OutputStream(samplerate=audio_fs, channels=1, dtype='float32')
stream.start()

# 3. Real-time Quadrature Demodulation and Streaming Loop
try:
    while True:
        # Buffer capture: Pull high-speed Complex IQ samples from PlutoSDR
        hardware memory
        rx_samples = sdr.rx()

        # Baseband FM Demodulation: Differentiate the unwrapped phase
        variations over time
        # This acts as a mathematical discriminator retrieving audio from
        the complex phase domain
        demodulated_raw = np.diff(np.unwrap(np.angle(rx_samples)))

        # Downsample the 1 MHz raw data rate down to standard 44.1 kHz
        audio speed
        audio_out = demodulated_raw[::decimation_rate]

        # Amplify the demodulated audio signal to pierce through the
        channel white noise
        audio_out = audio_out * 5.0
        audio_out = np.clip(audio_out, -1.0, 1.0) # Hard limiter to prevent
        digital clipping

        # Playback: Push the processed floating-point audio slice into
        Windows sound card queue
        stream.write(audio_out.astype(np.float32))

```

```
except KeyboardInterrupt:
    print("\n[RX] Muting the radio receiver and releasing hardware
    hooks...")
    stream.stop()
    stream.close()
```

4. Experimental Results & Troubleshoots

During development, multiple technical bottlenecks regarding the Windows software environment and the RF physical layer were systematically debugged and resolved:

1) Windows Path Environment Resolution (NoneType Error)

- Problem: Windows Python environment initially failed to locate the core hardware abstraction engine, throwing a `TypeError: argument of type 'NoneType' is not iterable` upon executing `import adi`.
- Solution: Installed the official standard binary package (`libiio-setup-v0.24-x64.exe`) to register the core libraries into the Windows system space. Integrated the `ctypes.windll.kernel32.SetDllDirectoryW(None)` interface in python to force dynamic runtime binding.

2) RF Over-the-Air Channel Optimization & Attenuation

- Problem: Operating under wireless bar antennas resulted in low Signal-to-Noise Ratio (SNR), causing severe white noise distortion due to multi-path fading within the indoor workspace.
- Solution: Replaced the wireless channel with a physical coaxial SMA cable link bridging the Linux TX1 and Windows RX1 ports. To mitigate hardware front-end saturation (clipping), the TX hardware attenuation was throttled to -20 dB and the RX gain adjusted to 20 dB, resulting in a crisp, crystal-clear real-time audio playback through the PC sound card.